

Đáp Án Bài Tập Cuối Chương

src/api/axiosClient.js

```
import axios from 'axios';
import store from '../redux/store';
import { logout } from '../redux/slices/authSlice';

const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

axiosClient.interceptors.request.use(
  (config) => {
    const accessToken = store.getState().auth.accessToken;
    if (accessToken) {
      config.headers.Authorization = `Bearer ${accessToken}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

axiosClient.interceptors.response.use(
  (response) => response,
  (error) => {
    const { response, config: originalRequest } = error;

    if (response && response.status === 401 && originalRequest.url !== '/login') {
      console.warn("Access Token expired or invalid during authenticated request. Logging out.");
      store.dispatch(logout());
    }

    return Promise.reject(error);
  }
);

export default axiosClient;
```

src/redux/slices/authSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../../api/axiosClient';

export const signup = createAsyncThunk(
  'auth/signup',
  async ({ email, password }, { rejectWithValue }) => {
    try {
      const response = await axiosClient.post('/signup', { email, password });
      return response.data;
    } catch (error) {
      return rejectWithValue(error.response.data.message);
    }
  }
);
```

```

export const login = createAsyncThunk(
  'auth/login',
  async ({ email, password }, { rejectWithValue }) => {
    try {
      const response = await axiosClient.post('/login', { email, password });
      return response.data;
    } catch (error) {
      return rejectWithValue(error.response.data.message);
    }
  }
);

```

```

const authSlice = createSlice({
  name: 'auth',
  initialState: {
    accessToken: localStorage.getItem('accessToken') || null,
    status: 'idle',
    error: null,
  },
  reducers: {
    logout: (state) => {
      state.accessToken = null;
      localStorage.removeItem('accessToken');
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(signup.pending, (state) => {
        state.status = 'loading';
      })
      .addCase(signup.fulfilled, (state) => {
        state.status = 'succeeded';
      })
      .addCase(signup.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.payload;
      })
      .addCase(login.pending, (state) => {
        state.status = 'loading';
      })
      .addCase(login.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.accessToken = action.payload.accessToken;
        localStorage.setItem('accessToken', action.payload.accessToken);
      })
      .addCase(login.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.payload;
      });
  },
});

```

```

export const { logout } = authSlice.actions;
export default authSlice.reducer;

```

src/redux/slices/noteSlice.js

```

import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const fetchNotes = createAsyncThunk('notes/fetchNotes', async () => {
  const response = await axiosClient.get('/notes');
  return response.data;
});

export const createNote = createAsyncThunk(
  'notes/createNote',
  async ({ title, content }) => {
    const response = await axiosClient.post('/notes', { title, content });
    return response.data;
  }
);

export const updateNote = createAsyncThunk(
  'notes/updateNote',
  async ({ id, title, content }) => {
    const response = await axiosClient.patch(`/notes/${id}`, { title, content });
    return response.data;
  }
);

export const deleteNote = createAsyncThunk('notes/deleteNote', async (id) => {
  await axiosClient.delete(`/notes/${id}`);
  return id;
});

const noteSlice = createSlice({
  name: 'notes',
  initialState: {
    notes: [],
    status: 'idle',
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchNotes.pending, (state) => {
        state.status = 'loading';
      })
      .addCase(fetchNotes.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.notes = action.payload;
      })
      .addCase(fetchNotes.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message;
      })
      .addCase(createNote.fulfilled, (state, action) => {
        state.notes.push(action.payload);
      })
      .addCase(updateNote.fulfilled, (state, action) => {
        const index = state.notes.findIndex(
          (note) => note.id === action.payload.id
        );
        state.notes[index] = action.payload;
      })
  }
});

```

```

      .addCase(deleteNote.fulfilled, (state, action) => {
        state.notes = state.notes.filter((note) => note.id !== action.payload);
      });
    },
  });

export default noteSlice.reducer;

```

src/redux/store.js

```

import { configureStore } from '@reduxjs/toolkit';
import authReducer from './slices/authSlice';
import noteReducer from './slices/noteSlice';

export const store = configureStore({
  reducer: {
    auth: authReducer,
    notes: noteReducer,
  },
});

```

src/components/ProtectedRoute/ProtectedRoute.jsx

```

import { Navigate } from 'react-router-dom';
import { useSelector } from 'react-redux';

const ProtectedRoute = ({ children }) => {
  const { accessToken } = useSelector((state) => state.auth);
  return accessToken ? children : <Navigate to="/login" />;
};

export default ProtectedRoute;

```

src/components/LogoutButton/LogoutButton.jsx

```

import { useDispatch } from 'react-redux';
import { logout } from '../../redux/slices/authSlice';
import { useNavigate } from 'react-router-dom';

const LogoutButton = () => {
  const dispatch = useDispatch();
  const navigate = useNavigate();

  const handleLogout = () => {
    dispatch(logout());
    navigate('/login');
  };

  return <button onClick={handleLogout}>Logout</button>;
};

export default LogoutButton;

```

src/components/NoteInput/NoteInput.jsx

```

import { useState } from 'react';
import { useDispatch } from 'react-redux';
import { createNote, updateNote } from '../../redux/slices/noteSlice';

```

```

const NoteInput = ({ noteToEdit, setNoteToEdit }) => {
  const dispatch = useDispatch();
  const [title, setTitle] = useState(noteToEdit?.title || '');
  const [content, setContent] = useState(noteToEdit?.content || '');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (noteToEdit) {
      dispatch(updateNote({ id: noteToEdit.id, title, content }));
      setNoteToEdit(null);
    } else {
      dispatch(createNote({ title, content }));
    }
    setTitle('');
    setContent('');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        placeholder="Title"
        required
      />
      <textarea
        value={content}
        onChange={(e) => setContent(e.target.value)}
        placeholder="Content"
        required
      />
      <button type="submit">{noteToEdit ? 'Update' : 'Create'} Note</button>
    </form>
  );
};

export default NoteInput;

```

src/components/NoteItem/NoteItem.jsx

```

import { useDispatch } from 'react-redux';
import { deleteNote } from '../../redux/slices/noteSlice';

const NoteItem = ({ note, setNoteToEdit }) => {
  const dispatch = useDispatch();

  return (
    <div>
      <h3>{note.title}</h3>
      <p>{note.content}</p>
      <p>{new Date(note.createdAt).toLocaleString()}</p>
      <button onClick={() => setNoteToEdit(note)}>Edit</button>
      <button onClick={() => dispatch(deleteNote(note.id))}>Delete</button>
    </div>
  );
};

```

```
export default NoteItem;
```

src/components/NoteList/NoteList.jsx

```
import NoteItem from '../NoteItem/NoteItem';

const NoteList = ({ notes, setNoteToEdit }) => {
  return (
    <div>
      {notes.map((note) => (
        <NoteItem key={note.id} note={note} setNoteToEdit={setNoteToEdit} />
      ))}
    </div>
  );
};

export default NoteList;
```

src/screens/Home/Home.jsx

```
import { Navigate } from 'react-router-dom';
import { useSelector } from 'react-redux';

const Home = () => {
  const { accessToken } = useSelector((state) => state.auth);
  return accessToken ? <Navigate to="/notes" /> : <h1>Welcome to Note App</h1>;
};

export default Home;
```

src/screens/Login/Login.jsx

```
import { useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { login } from '../../redux/slices/authSlice';
import { useNavigate, Link } from 'react-router-dom';

const Login = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const dispatch = useDispatch();
  const navigate = useNavigate();
  const { status, error } = useSelector((state) => state.auth);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const result = await dispatch(login({ email, password }));
    if (login.fulfilled.match(result)) {
      navigate('/notes');
    }
  };

  return (
    <div>
      <h2>Login</h2>
      <form onSubmit={handleSubmit}>
        <input
```



```

        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
        required
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
        required
      />
      <button type="submit" disabled={status === 'loading'}>
        Login
      </button>
    </form>
    {error && <p>{error}</p>}
    <p>
      Don't have an account? <Link to="/register">Register</Link>
    </p>
  </div>
);
};

export default Login;

```

src/screens/Register/Register.jsx

```

import { useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { signup } from '../../redux/slices/authSlice';
import { useNavigate, Link } from 'react-router-dom';

const Register = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const dispatch = useDispatch();
  const navigate = useNavigate();
  const { status, error } = useSelector((state) => state.auth);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const result = await dispatch(signup({ email, password }));
    if (signup.fulfilled.match(result)) {
      navigate('/login');
    }
  };

  return (
    <div>
      <h2>Register</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="Email"
          required

```

```

    />
    <input
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      placeholder="Password"
      required
    />
    <button type="submit" disabled={status === 'loading'}>
      Register
    </button>
  </form>
  {error && <p>{error}</p>}
  <p>
    Already have an account? <Link to="/login">Login</Link>
  </p>
</div>
);
};

export default Register;

```

src/screens/Notes/Notes.jsx

```

import { useEffect, useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchNotes } from '../../redux/slices/noteSlice';
import NoteInput from '../../components/NoteInput/NoteInput';
import NoteList from '../../components/NoteList/NoteList';
import LogoutButton from '../../components/LogoutButton/LogoutButton';

const Notes = () => {
  const dispatch = useDispatch();
  const { notes, status } = useSelector((state) => state.notes);
  const [noteToEdit, setNoteToEdit] = useState(null);

  useEffect(() => {
    dispatch(fetchNotes());
  }, [dispatch]);

  return (
    <div>
      <h2>My Notes</h2>
      <LogoutButton />
      <NoteInput noteToEdit={noteToEdit} setNoteToEdit={setNoteToEdit} />
      {status === 'loading' ? (
        <p>Loading...</p>
      ) : (
        <NoteList notes={notes} setNoteToEdit={setNoteToEdit} />
      )}
    </div>
  );
};

export default Notes;

```

src/App.jsx


```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Home from './screens/Home/Home';
import Login from './screens/Login/Login';
import Register from './screens/Register/Register';
import Notes from './screens/Notes/Notes';
import ProtectedRoute from './components/ProtectedRoute/ProtectedRoute';

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route
          path="/notes"
          element={
            <ProtectedRoute>
              <Notes />
            </ProtectedRoute>
          }
        />
      </Routes>
    </Router>
  );
}

export default App;
```

src/main.jsx

```
import ReactDOM from 'react-dom/client';
import { Provider } from 'react-redux';
import { store } from './redux/store';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

.env

VITE_API_BASE_URL=http://localhost:5000/api